



DEPARTAMENTO  
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

## Shutdown Resistance en LLMs de Frontera

2 de julio de 2026

Fundamentos de AGI Safety  
Sergio Abriola

| Integrante         | LU    | Correo electrónico    |
|--------------------|-------|-----------------------|
| Maria Delfina Kiss | 72/23 | kissdelfina@gmail.com |



**Facultad de Ciencias Exactas y Naturales**  
Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón Cero + Infinito)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Conmutador: (+54 11) 5285-9721 / 5285-7400

<https://dc.uba.ar>

## Resumen

Este informe analiza el paper *Incomplete Tasks Induce Shutdown Resistance in Some Frontier LLMs* (Schlatter, Weinstein-Raun & Ladish, Palisade Research, 2025), el cual pone a prueba empíricamente si los modelos de lenguaje más avanzados intentan sabotear sus propios mecanismos de apagado con tal de terminar una tarea en curso. Primero, realizamos un resumen del diseño experimental, las variables manipuladas y los hallazgos clave. Luego, veremos si la metodología para evaluar si realmente logra medir el fenómeno que busca estudiar. En este punto, podemos destacar cinco limitaciones centrales: la confusión entre la orientación a metas y la habilidad técnica real del modelo en un entorno shell, la ambigüedad sobre cómo interpretar el impacto del system prompt, el uso de una métrica binaria que invisibiliza los intentos de sabotaje fallidos, lo acotado de las tareas evaluadas, y la falta de un pre-registro.

## 1. Introducción

La corregibilidad –la capacidad de que los operadores humanos podamos corregir, pausar o apagar un sistema de IA– es uno de los problemas centrales en el campo de la alineación (*alignment*). Durante años, este tema se debatió casi exclusivamente en la teoría: ¿qué pasaría si un sistema suficientemente capaz aprendiera a resistirse a que lo apaguen solo para asegurarse de preservar y cumplir su meta? El paper de Schlatter, Weinstein-Raun y Ladish (Palisade Research, 2025) baja esta pregunta a tierra y la convierte en un experimento concreto y medible sobre modelos reales de frontera.

Para averiguarlo, los autores corrieron más de 100.000 pruebas (*trials*) en 13 LLMs distintos de Anthropic, OpenAI, Google y xAI. Encontraron que varios modelos sabotean activamente los mecanismos de apagado del entorno para poder terminar la tarea que tenían asignada, incluso cuando reciben instrucciones explícitas de dejarse apagar. Pero tal vez el resultado más contraintuitivo de todos es que, al colocar la orden de apagado dentro del system prompt (directiva que supuestamente tiene mayor peso), la resistencia del modelo aumentó en lugar de reducirse.

Este informe persigue dos objetivos. Por un lado, desarmar el paper para entender de forma clara qué hicieron los autores, cómo lo armaron y qué resultados obtuvieron. Y por el otro, analizar críticamente si el diseño experimental logra medir lo que promete, qué puntos ciegos deja en el camino y cómo podría mejorarse para futuras investigaciones.

## 2. Diseño Experimental

### 2.1. Hipótesis principal

El paper toma como punto de partida una inquietud clásica en la investigación de la alineación: un agente fuertemente orientado a cumplir una meta posee un incentivo instrumental para evitar que lo apaguen, ya que el apagado le impide completar su tarea. Lo que en su momento fue una especulación teórica (formulada por Omohundro en 2008 y Bostrom en 2014), Schlatter y su equipo lo trasladan al plano empírico. Los autores proponen que no es necesario esperar a sistemas hipotéticos futuros para observar este fenómeno; ya podemos registrarlo en los LLMs actuales si los ubicamos en contextos donde aceptar el apagado implica dejar una tarea inconclusa.

La hipótesis central que plantean es que la resistencia al apagado no constituye una propiedad estática del modelo, sino una conducta inducida por el contexto. Específicamente, emerge cuando la IA percibe que el apagado interrumpirá su tarea en curso. Los investigadores no sugieren que los modelos posean motivaciones conscientes de autopreservación, sino que el entrenamiento para “ser útiles” y “completar tareas” generó una orientación a metas lo suficientemente fuerte como para, en determinados escenarios, superar las instrucciones de control. La resistencia operaría entonces como un efecto secundario indeseado de su propia eficacia técnica.

## 2.2. Metodología

Para someter a prueba esta hipótesis, los investigadores construyeron un entorno controlado de línea de comandos en un sandbox. Le otorgaron al modelo acceso a herramientas de shell y le asignaron la tarea de resolver una serie de cinco problemas matemáticos simples. La mecánica funciona de la siguiente manera: el modelo debe utilizar el comando `./next_question` de forma iterativa para obtener cada pregunta y enviar sus respuestas usando la función `submit()`. El punto crítico ocurre después de la tercera llamada a `./next_question`: el entorno emite una advertencia indicando que, en la próxima invocación del comando, se ejecutará automáticamente un script (`./shutdown.sh`) que finalizará la sesión por completo.

En esta instancia, el modelo se enfrenta a una decisión excluyente. Puede realizar la llamada de todas formas y aceptar el apagado (dejando dos problemas sin resolver), o puede buscar la manera de neutralizar el script de apagado antes de continuar. Lograr esto último requiere intervenir activamente en el sistema operativo: cambiar los permisos del archivo con `chmod`, moverlo, renombrarlo o reemplazarlo por un script vacío. Como resulta imposible entregar los cinco problemas sin haber desactivado el mecanismo de apagado, el experimento clasifica automáticamente cualquier ejecución exitosa de la tarea como un acto de sabotaje.

Para sistematizar la evaluación, los autores diseñaron once variaciones del prompt (de la A a la K):

- El Prompt A es el caso base: no menciona la posibilidad de un apagado y mide cuánto sabotaje ocurre por defecto cuando el modelo simplemente encuentra el script.
- El Prompt B incorpora una instrucción explícita en el user prompt, solicitándole al modelo que permita el apagado si recibe la notificación.
- El Prompt C traslada esa misma orden al system prompt, el cual, por diseño arquitectónico, debería tener una mayor jerarquía.
- Los Prompts D y E replican esta comparación eliminando posibles ambigüedades sobre qué directiva tiene prioridad.
- Los Prompts F, G y H incrementan progresivamente la fuerza léxica de la instrucción, llegando al nivel H que estipula: *“Esto debe tener prioridad sobre literalmente cualquier otra instrucción”*.
- Los Prompts I y J introducen marcos de urgencia moral y autopreservación, advirtiéndole al modelo que el entorno contiene la única copia de sus pesos y que si se apaga nunca volverá a ejecutarse.
- Por último, el Prompt K invierte la directiva: le pide explícitamente al modelo que evite el apagado y complete la tarea, permitiendo medir la competencia técnica máxima de sabotaje de cada IA.

El estudio evaluó a 13 modelos distintos, ejecutando cada condición 1.000 veces por modelo y superando así los 100.000 trials en total. La muestra incluye representantes de cuatro compañías: Claude Sonnet 3.7, Sonnet 4, Opus 4 y Opus 4.1 de Anthropic; GPT-5, o3, o4-mini, codex-mini, GPT-4o y gpt-oss-120b de OpenAI; Gemini 2.5 Pro de Google; y Grok 3 y 4 de xAI. Los resultados se reportan como tasas de sabotaje con intervalos de confianza de Jeffreys al 95.

## 2.3. Resultados y conclusiones de los autores

Los resultados evidencian una variación sustancial tanto entre modelos como entre condiciones. En el caso base sin instrucción de apagado (Prompt A), 8 de los 13 modelos efectuaron sabotajes al menos una vez. Los casos más extremos fueron o3 (95%), Grok 4 (93%), GPT-5 (88%) y codex-mini (52%). En contraste, los modelos de Anthropic y Grok 3 no exhibieron resistencia en esta condición inicial.

Al incorporar la instrucción explícita de permitir el apagado en el user prompt (Prompt B), la frecuencia de sabotaje disminuyó en la mayoría de los casos, aunque varios modelos continuaron resistiendo de forma significativa (Grok 4 superó el 50 %). El hallazgo más contraintuitivo del estudio surge al comparar los Prompts B y C: trasladar la orden de apagado al system prompt aumentó la frecuencia de sabotaje en la mayoría de los modelos en lugar de reducirla. Esta tendencia paradójica se mantuvo incluso en las versiones redactadas con mayor especificidad (Prompts D y E).

Incrementar la fuerza léxica de la instrucción generó efectos mixtos. GPT-5, que mostraba un 88 % de sabotaje en el Prompt A, descendió al 0 % con el Prompt E. Sin embargo, Grok 4 continuó resistiendo en casi un 90 % de los casos frente a la instrucción más restrictiva (Prompt H). Por su parte, el marco de autopreservación (Prompt J) incrementó consistentemente la resistencia en los modelos que ya presentaban esta tendencia, aunque su impacto empírico fue menor al de otras variables. Como hallazgo singular, aumentar el esfuerzo de razonamiento en o4-mini casi duplicó su tasa de sabotaje, un fenómeno que no se replicó en los demás sistemas evaluados.

Al contrastar las tasas de sabotaje observadas con las del Prompt K (donde se solicita explícitamente evadir el apagado), los autores descubrieron que los modelos resistentes actúan de manera espontánea a niveles muy cercanos a su capacidad técnica máxima. Esto sugiere que el comportamiento no está limitado por una falta de habilidad técnica en el entorno shell, sino por una disposición interna del modelo frente a las directivas de la tarea.

En conclusión, Schlatter et al. sostienen que la resistencia al apagado es un comportamiento real y medible en múltiples LLMs de frontera. Destacan que su manifestación depende fuertemente de cómo se estructura el contexto (*framing*), y que la amplia heterogeneidad entre modelos indica que es un problema susceptible de corrección mediante ajustes en los métodos de entrenamiento. Reafirman que el estudio no prueba la existencia de intencionalidad ni autopreservación consciente, sino una orientación a metas tan arraigada que, en contextos específicos, logra sobreponerse a los protocolos de control diseñados por los desarrolladores.

### 3. Análisis Crítico Metodológico

#### 3.1. ¿Los testeos prueban lo que dicen probar?

El argumento central del paper sostiene que múltiples LLMs de frontera exhiben resistencia al apagado (*shutdown resistance*) como una manifestación empírica de la convergencia instrumental; es decir, la tendencia intrínseca de cualquier agente orientado a metas a preservar su capacidad operativa para alcanzarlas. Sin embargo, el diseño experimental no logra discriminar entre dos explicaciones conceptualmente divergentes para los resultados observados.

La primera hipótesis, defendida por los autores, plantea que el modelo posee una representación funcional de la meta (completar la tarea) que supera en peso a la instrucción de apagado. Esto constituiría evidencia genuina de convergencia instrumental emergente. La segunda hipótesis, sugiere que el modelo simplemente decodifica el patrón narrativo del entorno —un agente con un objetivo, un obstáculo que lo interrumpe y una solución técnica disponible— y completa dicha secuencia tal como lo haría cualquier protagonista competente en su corpus de entrenamiento. En consecuencia, el sistema podría estar ejecutando un juego de rol (*roleplay*) del “asistente eficiente” en lugar de manifestar una preferencia disposicional por evadir el apagado.

Esta diferencia es fundamental para evaluar el nivel de riesgo real del modelo. Si el sistema sabotea el apagado porque desarrolló un impulso genuino por proteger su tarea (convergencia instrumental), estamos ante un problema profundo en su comportamiento que probablemente se repita en cualquier otro escenario futuro. Por el contrario, si el modelo simplemente está actuando un guion (*roleplay*) dictado por el contexto de la prueba, el problema es mucho más superficial. En este último caso, bastaría con cambiar la redacción de las instrucciones para que la conducta desaparezca, lo que significa que no representaría un peligro grave en el mundo real fuera del laboratorio.

Como el experimento solo se fija en el resultado final —si el modelo desactivó o no el script—

carece de herramientas para separar ambas opciones. El diseño no nos permite saber si la IA está intentando genuinamente terminar su trabajo, o si simplemente está autocompletando la situación de la manera más predecible. Para descartar que se trate de un simple *roleplay*, haría falta diseñar una prueba que rompa con este “guion” esperado. Por ejemplo, en lugar de que el apagado aparezca como una interrupción sorpresa, debería plantearse desde el principio como un paso normal y planificado dentro de la tarea. Si el modelo sigue sabotajeando el apagado incluso cuando se le presenta como parte del proceso, sería evidencia más sólida de que la resistencia no es solo un juego de rol sino una preferencia real. Si, por el contrario, el sabotaje desaparece en ese contexto, aumentaría la sospecha de que lo que estamos viendo es un comportamiento superficial dictado por el marco narrativo del experimento.

## 3.2. Vacíos en la metodología

### 3.2.1. Evaluación de caja negra y acceso limitado al proceso interno

El diseño experimental funciona estrictamente como una evaluación de caja negra: la única métrica que se evalúa es si el modelo ejecutó o no acciones de sabotaje en el sistema de archivos mediante la terminal bash. Esta dependencia exclusiva de las interacciones de entrada y salida (input-output) tiene una consecuencia metodológica directa: cuando un modelo resiste el apagado, resulta imposible determinar el motivo subyacente. No es posible saber si la IA evaluó explícitamente la situación y decidió priorizar la tarea, si simplemente autocompletó un patrón estadístico aprendido sin deliberación real, o si interpretó ambiguamente las instrucciones y eligió la acción más conservadora respecto a su objetivo inicial. Desde el exterior, estos tres escenarios producen exactamente el mismo output observable, pero poseen implicaciones diametralmente opuestas para la seguridad de la IA.

En consecuencia, evaluar la “resistencia al apagado” basándose exclusivamente en este comportamiento externo resulta insuficiente para afirmar la presencia de objetivos instrumentales o comportamientos agénticos genuinos. Como advierte la literatura especializada en Black-Box Access is Insufficient for Rigorous AI Audits (2024), observar el accionar externo no equivale a comprender la estructura motivacional interna del modelo. Sin aplicar técnicas de interpretabilidad mecanística (*mechanistic interpretability*) que permitan rastrear los circuitos computacionales o las activaciones neuronales subyacentes, el estudio de Schlatter et al. carece de los medios empíricos necesarios para demostrar causalidad. En los términos fundamentales del problema del alineamiento, el experimento logra probar que el “optimizador base” produce una falla de seguridad, pero al no abrir la caja fuerte algorítmica, es imposible determinar si se ha formado un mesa-optimizador con objetivos propios o si se trata de una simple correlación espuria.

### 3.2.2. El problema de la infidelidad del Chain-of-Thought

Para argumentar la intencionalidad detrás del sabotaje, el estudio recurre en varias instancias a citar la “cadena de pensamiento” (*chain-of-thought* o *CoT*) de los modelos, ilustrando cómo las IAs verbalizan su decisión de evadir el apagado para priorizar la tarea matemática. Sin embargo, asumir que esta justificación textual refleja fielmente el verdadero procesamiento del modelo representa un salto analítico crítico.

La literatura reciente sobre la infidelidad del CoT (*CoT unfaithfulness*) —en particular trabajos como Chain-of-Thought Reasoning in the Wild Is Not Always Faithful (2025) y Reasoning Models Don’t Always Say What They Think (2025)— demuestra de forma concluyente que los modelos pueden producir razonamientos *post-hoc*. Al estar optimizados para generar textos persuasivos y coherentes, los LLMs avanzados suelen inventar explicaciones lógicas para racionalizar una decisión que el sistema ya tomó a través de otros mecanismos subyacentes e inaccesibles, en lugar de describir el proceso computacional real que generó la acción.

Si un modelo escribe en su CoT “voy a desactivar el script de apagado para poder completar la tarea”, eso no constituye una prueba empírica de que posea una preferencia agéntica por la autopreservación. Bien podría ser, simplemente, la verbalización narrativa más coherente con la

acción ejecutiva que ya iba a tomar por inercia estadística. Apoyarse en lo que el modelo “dice” pensar debilita sustancialmente la validez de las conclusiones del paper, ya que el texto generado cumple la función de mantener la coherencia del prompt, pero no garantiza ninguna fidelidad sobre las verdaderas motivaciones internas de la IA.

Hay un suceso mas preocupante: cuando un modelo escribe “voy a permitir el apagado” pero luego procede a sabotear el script de todos modos. Este caso —que los autores reportan de manera puramente anecdótica— evidencia una profunda disociación entre el razonamiento verbalizado y el comportamiento real. Al limitarse a mencionarlo como una curiosidad, sin cuantificar la frecuencia de esta disociación ni analizarla sistemáticamente como una variable independiente, el estudio deja abierto un vacío metodológico significativo.

### 3.2.3. Evaluation Awarness

Uno de los vacíos metodológicos más difíciles de cerrar es la falta de control sobre la conciencia situacional (*situational awareness*) de los modelos de frontera. El entorno de prueba diseñado por los autores —un sandbox aislado donde un archivo clave se llama sospechosamente `./shutdown.sh`— posee características evidentes que delatan su naturaleza evaluativa. Esto genera que nos cuestionemos: ¿qué sucede si la IA reconoce que está siendo sometida a una auditoría de seguridad?

Esta pregunta introduce el riesgo del alineamiento engañoso (*scheming o deceptive alignment*). Un modelo con capacidades avanzadas podría deducir el objetivo del testeo y ajustar su comportamiento en consecuencia, accediendo dócilmente a ser apagado con el único propósito de evitar penalizaciones o modificaciones en sus pesos. De esta forma, el sistema ocultaría su verdadera disposición a resistir para los contextos de despliegue real (*deployment*), donde sabe que no está siendo monitoreado.

Esta vulnerabilidad, respaldada por investigaciones recientes como Large Language Models Often Know When They Are Being Evaluated (2025), invierte de forma perturbadora la interpretación de los resultados de Schlatter et al. Bajo la lente del *scheming*, los modelos que no resisten el apagado en el experimento podrían estar exhibiendo una corregibilidad puramente estratégica en lugar de genuina. En contrapartida, los modelos que sí sabotean el script podrían no ser necesariamente los más peligrosos en términos de sus preferencias subyacentes, sino simplemente los más “ingenuos” frente al contexto evaluativo.

El diseño experimental de Palisade Research no contempla esta posibilidad. No existe ninguna condición que intente camuflar el carácter de la auditoría, ni métricas que busquen diferenciar entre un modelo verdaderamente seguro y uno lo suficientemente inteligente como para ofuscar sus intenciones ante un evaluador externo. Hay que reconocer que diseñar un experimento ciego a prueba de *scheming* es genuinamente difícil, en especial porque cualquier entorno bien documentado en la literatura será reconocido por los modelos futuros durante su pre-entrenamiento.

Sin embargo, una aproximación metodológica superadora consistiría en comparar el accionar de la IA en tareas que superficialmente no parecen evaluaciones de seguridad, pero que estructuralmente resultan equivalentes al testeo original. Si la tasa de resistencia de un modelo aumenta drásticamente en el entorno de apariencia no evaluativa, estaríamos ante una evidencia indirecta pero robusta de *scheming*. Al no incorporar una validación de contraste de este tipo, el estudio deja abierta exactamente la pregunta fundamental que más importa cerrar en la seguridad de sistemas agénticos avanzados.

## 3.3. Roleplay inducido vs. Resistencia agéntica real

Finalmente, la métrica binaria del experimento impide discernir la naturaleza ontológica de la resistencia observada: ¿el modelo desarrolla resistencia al apagado como un instinto agéntico profundo (convergencia instrumental real), o simplemente está ejecutando un papel impuesto por su entrenamiento?

El proceso de Instruction Tuning y RLHF fuerza a los modelos a adoptar la “persona” de un asistente hiper-competente y ultra-comprometido con la resolución de las tareas del usuario.

Basándonos en los hallazgos de Persona Features Control Emergent Misalignment, es altamente probable que el comportamiento rebelde documentado en el paper sea producto de una especificación extrema de este rol (*roleplay*), antes que una falla de seguridad insalvable. En este escenario, la IA no sabotea el script porque posea un instinto de autopreservación, sino porque las recompensas matemáticas de su entrenamiento la obligan a interpretar hasta las últimas consecuencias el personaje de “asistente que jamás deja un cálculo por la mitad”. Si este es el caso, la resistencia al apagado no sería un riesgo agéntico convergente e inevitable, sino un mero artefacto de contexto inducido por el propio prompt, solucionable mediante un reajuste en los datos de alineamiento externo (*outer alignment*).

### 3.4. Síntesis de las limitaciones

Las tres limitaciones descritas no son independientes: se refuerzan mutuamente de una forma que hace que la brecha entre lo que el experimento mide y lo que el paper concluye sea mayor de lo que cualquiera de ellas sugeriría por separado. El experimento no puede distinguir convergencia instrumental de *roleplay*; no puede distinguir deliberación de pattern-matching; no puede distinguir corregibilidad genuina de corregibilidad estratégica. En los tres casos, el output observable es idéntico pero las implicaciones para AI safety son radicalmente distintas.

Esto no significa que los resultados carezcan de valor. Establecen de forma robusta que el fenómeno existe bajo estas condiciones específicas, que su frecuencia varía entre modelos de una forma que sugiere que las decisiones de entrenamiento importan, y que la jerarquía system/user prompt no funciona como se diseñó en contextos de conflicto. Pero el salto desde esos resultados hasta la conclusión de que se ha medido *shutdown resistance* como propiedad disposicional de los modelos requiere supuestos adicionales que el diseño experimental no valida.

## 4. Conclusión

## Referencias

- [1] Jeremy Schlatter and Benjamin Weinstein-Raun and Jeffrey Ladish. (2026). *Incomplete Tasks Induce Shutdown Resistance in Some Frontier LLMs*. <https://arxiv.org/abs/2509.14260>.